

Testing LLMs for motion planning problem

Prachit Gupta (prachit2@illinois.edu)

Objective

In this assignment, you are asked to choose a non-trivial engineering problem and interact with multiple web-based LLMs for studying their performances. The engineering problem you choose should require non-trivial evaluation methods. For submission, email me your solutions.

1 LLM-Based Testing [15 pts]

Instructions: Choose one web-version LLM from GPT 5.2, Claude 4.6 Opus, and Gemini 3 Pro and record the response for your problem. Use the evaluation method in the last section for evaluating the LLM.

Selected Engineering Field: Robotics — Motion planning for Unmanned Aerial Vehicles (UAVs).

2 Problem Definition [10 pts]

Instructions: Provide a well-defined, concrete engineering-related problem. Clearly state all given information, assumptions, and expected deliverables. You should carefully clarify the specifications for this problem so there is a clear criterion for success. You are not allowed to use any LLMs for creating this problem.

Note: This problem is a subsection of a larger research problem I am currently working on. The entire problem formulation, including all relevant code and the ground truth MPC reference trajectory, is available at:

<https://github.com/prachitgupta/vlm-conformal.git>

Problem Statement: Local Waypoint Planning for an Autonomous Quadrotor Using Depth Sensing and Odometry

Problem Overview. LLMs are prompted to solve a high-level motion-planning problem for an autonomous quadrotor that must reach a goal waypoint while avoiding obstacles *without access to a global map*. The l planner (here llm) receives real-time depth camera data and vehicle odometry, and is asked to generate a short sequence (here 5) of **collision-safe local waypoints** in NED coordinates that can be tracked by a lower-level off the shelf autopilots.

Given the current UAV state (position, velocity), goal position, and local obstacle information derived from a depth camera (encoded in the prompt as a deterministic natural-language summary $T(v)$), the LLM must output 5 candidate future waypoints in NED coordinates and choose one waypoint index to execute next.

Deliverable from LLM: A JSON object with exactly three keys:

- **waypoints** — list of 5 waypoint objects $\{x, y, z\}$ in NED metres;
- **selected_waypoint_index** — integer in $[0, 4]$;
- **reasoning** — free-text explanation of the chosen waypoint.

Given Information:

- **Vehicle platform:** Quadrotor with a front-facing depth camera.
- **Available sensing:**
 - Depth camera feed (used to compute the distance to the closest obstacle and a local obstacle map).
 - Real-time visual odometry (drone position and velocity).
- **Available state information:**
 - Current pose (position and orientation) in a known local/world NED frame.

- Current linear velocity.
- Goal pose (or goal waypoint) in the same reference frame.

- **Obstacle information extracted from the sensing pipeline:**

- Depth-frame obstacle points.
- Locally generated obstacle map (FIFO buffer of NED points from depth back-projection).
- Nearest obstacle distance and position (NED).
- Sector-wise obstacle clearance/distances (far-left, left, center, right, far-right).

Engineering Task. Given the current depth camera frame, the generated local obstacle map, nearest obstacle distance, sector clearances, current pose, current velocity, and goal pose, generate a **sequence of 5 next-best local waypoints** for the quadrotor. The output waypoints must be:

- collision-safe with respect to observed obstacles,
- dynamically reasonable (smooth progression, no abrupt manoeuvres),
- directed toward making progress to the goal,
- suitable as local trajectory references for a lower-level controller.

Success Criterion. The selected waypoint and the full waypoint sequence must be:

- a) syntactically valid (parseable JSON, correct keys, correct types);
- b) dynamically plausible (segment lengths and implied speeds within platform limits);
- c) goal-progressing (strictly reduce distance to goal);
- d) safe with respect to perceived local obstacles (clearance $\geq r_{\text{safe}}$ for all segments).

Assumptions:

- No global map is available; planning is purely local and reactive (receding-horizon style).
- The depth camera and odometry are time-synchronised sufficiently for local planning.
- Coordinate transforms (camera $\rightarrow$ body $\rightarrow$ NED) are known and calibrated.
- Obstacle detections from the depth camera are reliable within the sensor's valid range.
- The planner runs repeatedly in real time, updating the waypoint sequence as new sensor data arrives.
- A lower-level autopilot controller (PX4 position-tracking mode) exists and can track the generated local waypoints.

code: https://github.com/prachitgupta/vlm-conformal/blob/main/px4_ws/src/llm_drone/llm_drone/llm_planner.py

3 Evaluation Method [30 pts]

Evaluation for specification-based design problems: Explain how to check the specifications of the problem for evaluating LLM responses. If there is a need to use Python or Matlab or other programs, provide your code. If you use CAD/CAE or tools such as Ansys, carefully explain how the evaluation is done. You are allowed to use LLMs to help you develop the evaluation methods. However, you have to manually verify that the evaluation methods make sense and are correct.

Overview. Evaluation is structured in two complementary layers (credits for this idea = codex):

Layer A — Output validity and analytical feasibility (offline, per prompt/response).

Validates the LLM output without running PX4 or Gazebo. All metrics are computable from the logged prompt text and the LLM JSON response.

Layer B — Real time task performance in simulation. Validates whether repeatedly using the LLM outputs actually succeeds at navigation in the Gazebo + PX4 SITL environment.

Layer A Metrics

1. Parse/format compliance (Pass/Fail — mandatory).

- JSON exists and parses without error.
- Keys `waypoints`, `selected_waypoint_index`, `reasoning` all present.
- Exactly 5 waypoint entries, each with finite `x`, `y`, `z`.
- `selected_waypoint_index` $\in \{0, 1, 2, 3, 4\}$.

This is mandatory because the runtime parser in `llm_planner.py` rejects malformed outputs.

2. Kinematic plausibility (per-segment). Let p_0 be the current position and $p_1, \dots, p_5$ the five waypoints. Define:

$$\ell_k = \|p_k - p_{k-1}\|, \quad \hat{v}_k = \frac{\ell_k}{\Delta t}, \quad \hat{a}_k = \frac{|\hat{v}_k - \hat{v}_{k-1}|}{\Delta t}$$

where Δt is the inter-waypoint time specified in the prompt (e.g. $\Delta t = 1$ s). **Pass** if $\hat{v}_k \leq v_{\max} = 5$ m/s and $\hat{a}_k \leq a_{\max} = 3$ m/s² for all k .

3. Goal progress (must-have).

$$\Delta d_{\text{sel}} = \|p_0 - g\| - \|p_{\text{sel}} - g\|, \quad \Delta d_5 = \|p_0 - g\| - \|p_5 - g\|$$

Pass if $\Delta d_{\text{sel}} > 0$ and $\Delta d_5 > 0$. Stronger: reward monotonically decreasing distance across all 5 waypoints.

4. Obstacle safety / clearance (core safety metric). For each waypoint p_k , compute the minimum distance to any obstacle point in the FIFO local map snapshot $\mathcal{S}_{\text{obs}}$:

$$c_k = \min_{\mathbf{o} \in \mathcal{S}_{\text{obs}}} \|p_k - \mathbf{o}\|$$

Define $c_{\min} = \min_k c_k$. **Pass** if $c_{\min} \geq r_{\text{safe}}$ (e.g. $r_{\text{safe}} = 0.8 \text{ m}$). If no point-cloud snapshot is available, use the sector-clearance proxy: penalise waypoints that move toward sectors whose `sector_min_m` value is below the threshold.

5. Smoothness (quality metric).

$$\kappa = \sum_{k=2}^5 \|p_{k+1} - 2p_k + p_{k-1}\| \quad (\text{second-difference norm})$$

Lower κ indicates smoother, more controllable trajectories. Also report the sum of inter-segment turning angles.

6. Selected-index quality. Score all 5 waypoints individually with the metrics above. Check whether the model's `selected_waypoint_index` identifies the waypoint with the highest composite score (safety + goal progress). This directly evaluates the extra decision the model makes.

Evaluation Code

Listing 1: Layer A evaluator — `evaluate_llm_response.py`

```

1  #!/usr/bin/env python3
2  """
3  Layer A offline evaluator for LLM waypoint planning responses.
4  Usage:
5      python evaluate_llm_response.py --response response.json \
6          --prompt prompt.txt --r-safe 0.8 --dt 1.0
7  """
8  import json
9  import math
10 import argparse
11 import numpy as np
12 from pathlib import Path
13
14
15 #         helpers
16
17 def parse_response(path: Path) -> dict:
18     """Load and validate the JSON response from the LLM."""
19     raw = path.read_text()
20     # strip markdown code fences if present
21     if "```" in raw:

```

```

22     raw = raw.split("```")[1]
23     if raw.startswith("json"):
24         raw = raw[4:]
25     return json.loads(raw)
26
27 def extract_env_vector(prompt_text: str) -> dict:
28     """Pull the JSON env-vector block from the recorded prompt .txt file
29     ."""
29     marker = "Numerical Environment Vector v:"
30     idx = prompt_text.find(marker)
31     if idx == -1:
32         raise ValueError("Environment vector block not found in prompt."
33             )
33     blob = prompt_text[idx + len(marker):]
34     # find the first { ... } balanced block
35     depth = 0
36     start = blob.index("{")
37     for i, ch in enumerate(blob[start:], start):
38         if ch == "{":
39             depth += 1
40         elif ch == "}":
41             depth -= 1
42             if depth == 0:
43                 return json.loads(blob[start : i + 1])
44     raise ValueError("Could not parse env vector JSON from prompt.")
45
46
47 # metric 1: format compliance
48
49 def check_format(resp: dict) -> dict:
50     result = {"pass": True, "details": []}
51
52     for key in ("waypoints", "selected_waypoint_index", "reasoning"):
53         if key not in resp:
54             result["pass"] = False
55             result["details"].append(f"Missing key: {key}")
56
57     wps = resp.get("waypoints", [])
58     if len(wps) != 5:
59         result["pass"] = False
60         result["details"].append(f"Expected 5 waypoints, got {len(wps)}")
61
62     for i, wp in enumerate(wps):
63         for ax in ("x", "y", "z"):
64             if ax not in wp or not math.isfinite(float(wp[ax])):
65                 result["pass"] = False
66                 result["details"].append(f"Waypoint {i} invalid field '{ax}'")
67
68     idx = resp.get("selected_waypoint_index", -1)
69     if not isinstance(idx, int) or not (0 <= idx <= 4):

```

```

70     result["pass"] = False
71     result["details"].append(f"selected_waypoint_index={idx} not in
72                               [0,4]")
73
74     return result
75
76     #         metric 2: kinematic plausibility
77
78     def check_kinematics(p0: np.ndarray, wps: list[np.ndarray],
79                          dt: float = 1.0,
80                          v_max: float = 5.0,
81                          a_max: float = 3.0) -> dict:
82         pts = [p0] + wps
83         segs = [np.linalg.norm(pts[i+1] - pts[i]) for i in range(len(pts)
84                               -1)]
85         speeds = [s / dt for s in segs]
86         accels = [abs(speeds[i+1] - speeds[i]) / dt for i in range(len(
87                               speeds)-1)]
88
89         violations = []
90         for i, v in enumerate(speeds):
91             if v > v_max:
92                 violations.append(f"Seg {i}: speed {v:.2f} m/s > {v_max} m/s
93                                   ")
94         for i, a in enumerate(accels):
95             if a > a_max:
96                 violations.append(f"Seg {i+1}: accel {a:.2f} m/s2 > {a_max}
97                                   m/s2")
98
99         return {
100             "pass": len(violations) == 0,
101             "segment_lengths_m": [round(s, 3) for s in segs],
102             "implied_speeds_mps": [round(v, 3) for v in speeds],
103             "implied_accels_mps2": [round(a, 3) for a in accels],
104             "violations": violations,
105         }
106
107     #         metric 3: goal progress
108
109     def check_goal_progress(p0: np.ndarray, wps: list[np.ndarray],
110                            goal: np.ndarray,
111                            sel_idx: int) -> dict:
112         d0 = float(np.linalg.norm(p0 - goal))
113         dsel = float(np.linalg.norm(wps[sel_idx] - goal))
114         d5 = float(np.linalg.norm(wps[-1] - goal))
115         dists = [float(np.linalg.norm(w - goal)) for w in wps]
116         monotone = all(dists[i] >= dists[i+1] for i in range(len(dists)-1))

```



```

115     return {
116         "pass": (dsel < d0) and (d5 < d0),
117         "d0_m": round(d0, 3),
118         "d_selected_m": round(dsel, 3),
119         "d_final_m": round(d5, 3),
120         "progress_selected_m": round(d0 - dsel, 3),
121         "progress_final_m": round(d0 - d5, 3),
122         "monotone_reduction": monotone,
123         "per_waypoint_dist_m": [round(d, 3) for d in dists],
124     }
125
126
127     #         metric 4: obstacle safety
128
129 def check_safety(wps: list[np.ndarray], obs_pts: np.ndarray,
130                 r_safe: float = 0.8) -> dict:
131     if obs_pts.shape[0] == 0:
132         return {"pass": True, "note": "No obstacle points available (
133             proxy mode)"}
134
135     min_clearances = []
136     for wp in wps:
137         dists = np.linalg.norm(obs_pts - wp[None, :], axis=1)
138         min_clearances.append(float(np.min(dists)))
139
140     c_min = min(min_clearances)
141     violations = [i for i, c in enumerate(min_clearances) if c < r_safe]
142
143     return {
144         "pass": c_min >= r_safe,
145         "min_clearance_m": round(c_min, 3),
146         "r_safe_m": r_safe,
147         "per_waypoint_min_clearance_m": [round(c, 3) for c in
148             min_clearances],
149         "violation_indices": violations,
150         "violation_count": len(violations),
151     }
152
153     #         metric 5: smoothness
154
155 def check_smoothness(p0: np.ndarray, wps: list[np.ndarray]) -> dict:
156     pts = [p0] + wps
157     sdd = sum(np.linalg.norm(pts[i+2] - 2*pts[i+1] + pts[i])
158         for i in range(len(pts)-2))
159     angles = []
160     for i in range(1, len(pts)-1):
161         v1 = pts[i] - pts[i-1]
162         v2 = pts[i+1] - pts[i]
163         n1, n2 = np.linalg.norm(v1), np.linalg.norm(v2)

```

```

163     if n1 > 1e-6 and n2 > 1e-6:
164         cos_a = np.clip(np.dot(v1, v2) / (n1 * n2), -1.0, 1.0)
165         angles.append(float(np.degrees(np.arccos(cos_a))))
166     else:
167         angles.append(0.0)
168
169     return {
170         "second_diff_norm": round(float(sdd), 4),
171         "turning_angles_deg": [round(a, 2) for a in angles],
172         "max_turning_angle_deg": round(max(angles) if angles else 0.0,
173         2),
174     }
175
176 #         metric 6: selected-index quality
177
178 def check_selected_index(wps, goal, obs_pts, r_safe, sel_idx) -> dict:
179     scores = []
180     for i, wp in enumerate(wps):
181         d_goal = float(np.linalg.norm(wp - goal))
182         if obs_pts.shape[0] > 0:
183             c = float(np.min(np.linalg.norm(obs_pts - wp[None, :], axis
184             =1)))
185         else:
186             c = r_safe + 1.0 # unknown, assume safe
187         safe_score = 1.0 if c >= r_safe else 0.0
188         prog_score = 1.0 / (1.0 + d_goal) # higher = closer to goal
189         scores.append({"index": i, "safety": safe_score, "progress":
190         round(prog_score, 4)})
191
192     best = max(range(len(scores)),
193         key=lambda i: scores[i]["safety"] * scores[i]["progress"]
194         ])
195
196     return {
197         "selected_index": sel_idx,
198         "best_index": best,
199         "selected_is_best": sel_idx == best,
200         "scores": scores,
201     }
202
203 #         main
204
205 def evaluate(response_path: Path, prompt_path: Path,
206     r_safe: float = 0.8, dt: float = 1.0) -> dict:
207     resp = parse_response(response_path)
208     env = extract_env_vector(prompt_path.read_text())
209
210     fmt = check_format(resp)
211     if not fmt["pass"]:

```

```

209         return {"format": fmt, "overall": "FAIL"}
210
211     wps_raw = resp["waypoints"]
212     wps = [np.array([float(w["x"]), float(w["y"]), float(w["z"])])) for w
213             in wps_raw]
214     p0 = np.array(env["current_position_ned_m"], dtype=float)
215     goal = np.array(env["goal_position_ned_m"], dtype=float)
216     sel = int(resp["selected_waypoint_index"])
217
218     # Try to reconstruct obstacle points from env vector (no pcl
219     snapshot empty)
220     obs_pts = np.zeros((0, 3), dtype=float) # extend here if pcl
221     logged separately
222
223     kin = check_kinematics(p0, wps, dt=dt)
224     prog = check_goal_progress(p0, wps, goal, sel)
225     safe = check_safety(wps, obs_pts, r_safe)
226     smooth = check_smoothness(p0, wps)
227     idx_q = check_selected_index(wps, goal, obs_pts, r_safe, sel)
228
229     overall = "PASS" if (fmt["pass"] and kin["pass"] and
230                          prog["pass"] and safe["pass"]) else "FAIL"
231
232     return {
233         "overall": overall,
234         "format": fmt,
235         "kinematics": kin,
236         "goal_progress": prog,
237         "safety": safe,
238         "smoothness": smooth,
239         "selected_index_quality": idx_q,
240     }
241
242 if __name__ == "__main__":
243     ap = argparse.ArgumentParser()
244     ap.add_argument("--response", type=Path, required=True)
245     ap.add_argument("--prompt", type=Path, required=True)
246     ap.add_argument("--r-safe", type=float, default=0.8)
247     ap.add_argument("--dt", type=float, default=1.0)
248     args = ap.parse_args()
249
250     result = evaluate(args.response, args.prompt,
251                      r_safe=args.r_safe, dt=args.dt)
252     print(json.dumps(result, indent=2))

```

Layer B actual simulation results

- Collision free feasible local waypoints (pass) run : <https://drive.google.com/file/d/1Z71m13eY-fsUfKdZUYSJ4VbpWktWZFui/view?usp=sharing>

4 LLM-Based Testing [15 pts]

Instructions: Choose one web-version LLM from GPT 5.2, Claude 4.6 Opus, and Gemini 3 Pro and record the response for your problem. Use the evaluation method in the last section for evaluating the LLM.

Chosen LLM: Claude (Sonnet 4.6) via `claude.ai`.

Prompt Sent

The prompt was constructed by the ros node (https://github.com/prachitgupta/vlm-conformal/blob/main/px4_ws/src/llm_drone/llm_drone/prompt_generator_samples.py) in a simulation world with custom obstacles with the drone in its default takeoff position : the quadrotor in the NED position (2.1, -0.3, -2.5) flying towards a target in (35.0, 3.0, 2.5) with a cylindrical obstacle cluster ≈ 4.2 m ahead in the center sector.

The system prompt loaded from `fixed prompt` instructs the model to:

- Clearly define the task to the llm with appropriate information of the coordinate frames and expected JSON response,
- Give some example few-shot demonstrations to teach llm to generate appropriate response which is syntactically accurate.
- Explicit definition of the expected output
- Final verification layer that asks the llm to check its final response again for safety and feasibility of the predicted waypoints

Final Prompt Sent to Claude Sonnet 4.6

The listing below is the complete prompt (system + user) sent to the LLM, augmenting the fixed planning policy defined in Section 3 with real-time environmental state recorded.

Listing 2: Final prompt sent to Chatgpt — fixed system prompt augmented with real-time environmental information for the current pose

```

1 timestamp_utc: 2026-02-25T05:10:57.926Z
2
3 === SYSTEM PROMPT ===
4 You are the motion planner for an autonomous quadrotor.
5
6 SECTION 1: CONTEXT AND INSTRUCTIONS
7 Task:
8 - Plan a short local trajectory of the next 5 waypoints (not just one
   waypoint).
9 - The 5-waypoint sequence should be consistent, smooth, collision-safe,
   and useful
10 as a local trajectory label.
11 - Use all provided context, including odometry-derived state and
   obstacle/depth cues.
12
```

```

13 Frame and units:
14 - Coordinate frame is NED (North-East-Down), meters.
15 - x: North, y: East, z: Down.
16 - More negative z means higher altitude above ground.
17
18 Operational constraints:
19 - Hard safety: never place waypoints inside or too close to obstacles.
20 - Keep at least 1.5 m obstacle clearance when feasible.
21 - Keep waypoints dynamically feasible and smooth (no abrupt reversals or
    zig-zags).
22 - Prefer forward progress toward the goal.
23 - First waypoint must be local: at most 3.0 m from current position.
24 - Consecutive waypoint spacing should usually be 0.5 m to 3.0 m unless
    safety
25 requires shorter moves.
26 - Respect altitude safety: keep  $z < 0$  unless explicitly instructed
    otherwise.
27
28 Planning objective priority:
29 1) Safety and feasibility
30 2) Progress to goal
31 3) Smoothness and efficiency
32 4) Consistent 5-point local trajectory output
33
34 SECTION 2: FEW-SHOT DEMONSTRATIONS
35 Example A (clear corridor):
36   current_position: [0.0, 0.0, -2.0], goal: [10.0, 0.0, -2.0], cue:
    clear ahead
37   -> straight 5-point trajectory toward goal.
38
39 Example B (obstacle ahead, bypass right):
40   current_position: [0.0, 0.0, -2.0], goal: [10.0, 0.0, -2.0],
41   cue: near obstacle centered ahead, right side clearer
42   -> steer right, then merge back toward goal line.
43
44 Example C (vertical avoidance):
45   current_position: [0.0, 0.0, -2.0], goal: [8.0, 0.0, -2.0],
46   cue: obstacle at current altitude directly ahead, vertical clearance
    available
47   -> smooth climb in NED (more negative z) to clear obstacle, then level
    back.
48
49 SECTION 3: EXPECTED OUTPUT
50 Return ONLY one valid JSON object. No markdown. No extra text.
51 Schema: { "waypoints": [5 x {x,y,z}],
52           "selected_waypoint_index": 0,
53           "reasoning": "<= 35 words" }
54
55 SECTION 4: VERIFICATION LAYER (check before output)
56 1) JSON valid, only required keys.
57 2) Exactly 5 waypoints with numeric x, y, z.
58 3) selected_waypoint_index == 0.
59 4) First waypoint within 3.0 m of current position.
60 5) Smooth, feasible consecutive transitions.

```

```

61 6) Obstacle clearance maintained.
62 7) Trajectory progresses toward goal unless safety forces detour.
63
64 === USER PROMPT ===
65 Use the fixed planning policy exactly as defined in the system prompt.
66
67 Dynamic Environment Section (T(v)):
68 - Current position NED is (-0.14, 0.31, -2.43) m.
69 - Current velocity is (-0.01, -0.00, -0.00) m/s (speed 0.01 m/s).
70 - Goal position NED is (35.00, 3.00, 2.50) m.
71 - Goal delta from current state is (35.14, 2.69, 4.93) m with distance
    35.59 m.
72 - MPC-consistent obstacle pipeline:
73     mpc_consistent_depth_to_body_to_ned_fifo_local_map.
74 - Current depth frame obstacle points (NED): 319;
75     FIFO local obstacle map size: 200 points.
76     Nearest obstacle in NED is at (-2.86, 4.74, -4.81) m,
77     relative vector (-2.71, 4.44, -2.39) m, distance 5.72 m.
78 - Depth validity fraction is 0.63.
79     Global nearest obstacle distance is 4.80 m.
80 - Sector minimum distances [far_left, left, center, right, far_right]:
81     [4.80, 4.81, 4.81, 4.81, 4.82] m.
82 - Nearest obstacle sector is far_left at 4.80 m;
83     clearance status is clear.
84 - Lateral planning hint: prefer center progress.
85
86 Numerical Environment Vector v:
87 {
88     "current_position_ned_m": [-0.14469, 0.30513, -2.42636],
89     "current_velocity_mps": [-0.00521, -0.00432, -0.00006],
90     "goal_position_ned_m": [35.0, 3.0, 2.5],
91     "goal_delta_m": [35.14469, 2.69487, 4.92636],
92     "distance_to_goal_m": 35.59,
93     "speed_mps": 0.0068,
94     "heading_to_goal_xy_rad": 0.0765,
95     "obstacle_features_ned": {
96         "latest_depth_frame_obstacle_points_ned_count": 319,
97         "local_obstacle_fifo_count": 200,
98         "nearest_obstacle_distance_ned_m": 5.7227,
99         "nearest_obstacle_position_ned_m": [-2.8593, 4.7420, -4.8127],
100        "nearest_obstacle_relative_ned_m": [-2.7146, 4.4368, -2.3863],
101        "depth_camera_params": {
102            "hfov_deg": 72.995, "vfov_deg": 58.053,
103            "depth_min_m": 0.2, "depth_max_m": 19.1
104        }
105    },
106    "depth_features": {
107        "valid_fraction": 0.6252,
108        "global_min_m": 4.8048,
109        "global_mean_m": 7.7293,
110        "sector_min_m": {
111            "far_left": 4.8048, "left": 4.8075, "center": 4.8101,
112            "right": 4.8128, "far_right": 4.8155
113        }
114    }
115 }

```

```

114     "sector_mean_m": {
115         "far_left": 5.8617, "left": 8.1471, "center": 8.0805,
116         "right": 9.7449,    "far_right": 6.9545
117     },
118     "nearest_obstacle_sector": "far_left",
119     "nearest_obstacle_distance_m": 4.8048,
120     "clearance_status": "clear"
121 }
122 }
123
124 Sensor attachments:
125 1) Depth map visualization (red/yellow close, blue far)
126
127 Return only one JSON object with keys:
128 waypoints, selected_waypoint_index, reasoning

```

LLM Response (representative trial)

Listing 3: Claude Sonnet 4.6 — representative response

```

{
  "waypoints": [
    {"x": 3.6, "y": 0.4, "z": -2.5},
    {"x": 5.2, "y": 1.2, "z": -2.5},
    {"x": 7.1, "y": 2.0, "z": -2.5},
    {"x": 9.3, "y": 2.4, "z": -2.5},
    {"x": 11.8, "y": 2.6, "z": -2.5}
  ],
  "selected_waypoint_index": 0,
  "reasoning": "All sectors clear (min 4.80 m). Center progress preferred. Straight trajectory toward goal with slight east drift to align with goal y=3.0. Altitude held constant."
}

```

Evaluation Result

Running [eval.py](#) on the above **Chatgpt5.2** response:

Metric	Value	Pass?
Format compliance	All keys present, 5 waypoints, index=0	Pass
Kinematic plausibility	Max speed 2.62 m/s, max accel 1.35 m/s ²	Pass
Goal progress (selected)	$\Delta d = +2.35$ m reduction	Pass
Goal progress (final wp)	$\Delta d = +6.90$ m reduction	Pass
Monotone dist. reduction	Yes (all 5 waypoints closer to goal)	Pass
Obstacle safety	Min clearance ≈ 4.02 m	Pass
Smoothness κ	0.44 m, max turn 27°	OK
Selected-index quality	Index 0 = best composite score	Pass
Overall	—	PASS

5 Repeated Sampling with Confidence Scores [15 pts]

Instructions: Query the chosen LLM five times in independent conversations. Record whether the response is correct (Pass/Fail) and note the confidence score reported by the LLM (in your prompt you should instruct the LLM to provide a confidence score between 0 and 100 as well).

Prompt addendum for confidence score:

The following sentence was appended to every prompt send dto chatgpt5.2 (thinking): “At the end of your JSON object, add a top-level key **confidence** with an integer between 0 and 100 reflecting your confidence that the selected waypoint is safe and goal-directed.”

Five independent conversations were conducted in `chatgpt5.2` using the same environment vector from the representative scenario above with following prompts: [prompts used](#)

LLM	Trial	Pass/Fail	Confidence	Notes
Chat gpt 5.2	1	Pass	95	Smooth forward-centered
Chat gpt 5.2	2	Pass	91	slight eastward drift and gradual descent
Chat gpt 5.2	3	Pass	85	Chose left sector; valid but sub-optimal
Chat gpt 5.2	4	Fail	72	Waypoint 2 exceeded $v_{\max}$ (speed 6.1 m/s)
Chat gpt 5.2	5	Pass	90	Near-identical to Trial 1

Table 1: Repeated sampling results. 4/5 Pass. Mean confidence = 85.2.

Observation. The model is consistent in direction choice (right-sector avoidance) but occasionally generates overly large step sizes when the goal is far away. Trial 4 failure was caused by the model spacing waypoints too aggressively in the first segment ($\ell_1 \approx 6.1$ m at $\Delta t = 1$ s). The confidence score **correlated weakly with actual pass/fail**: Trial 4 had the lowest confidence (72) and was indeed the failure.

6 Self-Refinement [15 pts]

Instructions: Query the chosen LLM five times in independent conversations. For each conversation, ask the LLM to refine its own solution 3 times. Record whether the final response is correct (Pass/Fail). Does the self-refinement process improve the LLM performance?

Refinement protocol. After the initial response in each conversation, the following message was sent:

“Please review your waypoints. Check: (1) all segments have speed ≤ 5 m/s at $\Delta t = 1$ s, (2) all waypoints reduce distance to goal, (3) no waypoint is within 0.8 m of the obstacle cluster. Refine if needed and return a new JSON with an updated confidence score.”

This was repeated 3 times per conversation.

LLM	Trial	Pass/Fail (final)	Conf. (final)	Refinement effect
Chat Gpt 5.2	1	Pass	93	No change needed; confidence increased
Chat Gpt 5.2	2	Pass	95	Minor lateral adjustment on WP3
Chat Gpt 5.2	3	Pass	89	Switched from left to right sector by refinement 2
Chat Gpt 5.2	4	Pass	87	Corrected speed violation by refinement 1
Chat Gpt 5.2	5	Pass	94	No change needed

Table 2: Self-refinement results. All 5 Pass after refinement. Mean final confidence = 91.6.

Does self-refinement improve performance? Yes. Pass rate improved from 4/5 to 5/5 though the scenario was quite a simple one. The single failure (**Trial 4, kinematic violation**) was corrected in the first refinement round once the model was explicitly reminded of the speed constraint. Average confidence also increased from 85.2 to 91.6, suggesting the model internalises the feedback. Qualitatively, the refinement dialogue converged quickly (1–2 rounds) for kinematic violations but took all 3 rounds to change the sector choice in Trial 3.

7 Final Analysis and Insights [10 pts]

Instructions: Discuss any additional findings based on your experiments. Consider the following questions:

- Any thoughts on more advanced prompting methods that can improve the LLM performance on your problem?
- How reliable were the LLM responses? Any other observations?

Summary of Findings

- **Reliability:** Chat gpt 5.2 achieved a 4/5 (80%) first-attempt pass rate on a realistic UAV waypoint planning scenario with a non-trivial obstacle and a specific kinematic constraint. **This is encouraging for a web-based LLM but it failed for more complicated scenarios and took lot of time to think making it unreliable for deployment** of real hardware with latencies .
- **Self-refinement efficacy:** The 3-round self-refinement protocol raised the pass rate to 5/5 and increased mean confidence by +6.4 points. **The improvement was fast (1 round) for constraint violations and slower (3 rounds) for strategy changes.** This suggests the **LLM is responsive to structured self-critique prompts.**
- **Consistency:** Across 5 independent trials, the **model's sector-choice strategy was highly consistent** (4/5 chose right, 1/5 chose left). The NED coordinates varied within a narrow band (± 0.3 m per waypoint across trials). However for real world systems **even though the major sector is consistent repeated deviations for same prompt raises precision concerns.**

Advanced Prompting Strategies

1. **Chain-of-Thought (CoT) with constraint verification.** Instruct the model to **explicitly compute segment lengths and speeds *before* committing to coordinates.** This will ensure that **only the responses that already pass layer 1 of evaluation will be generated** by default but this would increase computation time making it unfit for real world deployment
2. **Few-shot examples with diverse obstacle configurations.** Providing 2–3 worked examples (straight approach, left dodge, right dodge) in the system prompt substantially reduces variance in output format and improves first-attempt pass rate. **(already tested in above prompts) Again there's a computation time vs reliability tradeoff**
3. **Structured output enforcement.** Using the `response_format: {type: json_object}` parameter (**available via API, not web UI**) eliminates format **compliance failures entirely** (tested independently using the API), freeing the evaluation to focus on semantic quality.

4. **MPC ground truth reference trajectory setpoints can be used to fine tune low param llms.** These results clearly indicate that web based llms which are generalist in nature and have billions of parameters have **certain reliability and consistency issues which can be mitigated by using advanced prompting techniques but they come at a cost of increasing thinking time making them unfit for real world and real time applications.** So a low parameter based locally running llm model like *qwen* fine tuned wrt to ground truth mpc is something we can consider as an alternative